Terminal-Bench on AgentCore

One image, one Runtime, one microVM per episode

Kaige Yang
Senior Applied Scientist, AWS

September 25, 2026

Agenda

00 Overview

The deck in one table; use case, isolation and design

01 Setup

218 Runtimes, the dataset, how one episode runs

02 The challenge: cold start

Why every episode pays a boot, and why slime can't hide it

03 The solution: look-ahead pre-open

Four mechanisms, the step boundary, sizing the window

04 Results

Smoke run, warm-start probe, takeaways and next steps

Appendix (from slide 18): references.

© 2026, Amazon Web Services, Inc. or its affiliates.

Look-ahead pre-open hid every boot; idle cost and sparse reward remain

Table 1 The deck in one table

Goal	RL-train Qwen3-4B (slime GRPO) on Terminal-Bench, every episode in its own AgentCore microVM
Challenge	Isolation needs a **fresh microVM per episode**, so every episode pays a **cold start**, in bursts, at every step
Solution	**Look-ahead pre-open:** replay slime's data order and boot the next 2 batches' sessions (P = 64) before slime asks; heartbeat keeps them alive (slide 10)
Result	Smoke run on AWS: 382 episodes, **0 infra errors**, `open_wait_ms` **p95 = 0** (no boot on the critical path). Cost: ≈ 95 % of microVM time idle; reward sparse (4 solves) (slide 14)
Warm-start probe	Pre-open cuts wait p50 774 → **5 ms** (own Runtime) / 1715 → **0 ms** (Code Interpreter), makespan −20 % / −15 %. Heartbeat keeps own-Runtime sessions past the idle timeout (3/3 vs 0/3) (slides 15–16)

Source: `code/demos/demo_cpu_tasks/terminalbench/` (smoke run 2026-09-23, warm-start probe 2026-09-25); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Terminal-Bench is the case Code Interpreter can’t cover: custom images at scale

Table 2 **What this demo covers**

Use case covered	Tasks that need **their own OS image** (packages, `/etc`, services): hundreds of custom images, one per task, routed per episode
Why this demo	The default Code Interpreter covers most literature tasks; this is the case it cannot: custom images **at scale**, where the cold start is paid per episode
Isolation	One episode = one **brand-new session** = one dedicated microVM, stopped (VM terminated, memory sanitized) when the episode ends. Nothing leaks between episodes or GRPO siblings
Design considerations	Strict image routing (unmapped task = error, no fallback) · look-ahead pre-open hides the boot · always-on heartbeat keeps pre-opened sessions past the 15-min idle timeout · retry on a new id; infra failures never scored as wrong answers

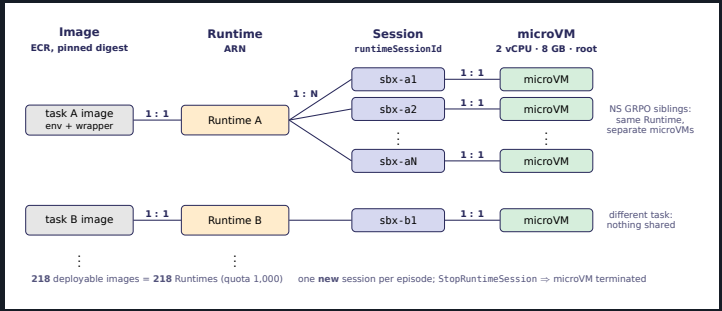
In plain English: every episode gets a brand-new computer built from its task’s image, and we boot those computers ahead of time so the trainer never waits.

Source: `code/demos/demo_cpu_tasks/terminalbench/`; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

218 tasks, 218 images, 218 Runtimes; every episode gets a new microVM

Figure 1 image 1:1 Runtime 1:N session 1:1 microVM



- **Each Terminal-Bench task is its own environment** (OS, packages, a deliberately broken state), so each task gets its own arm64 image and its own AgentCore Runtime: **218 Runtimes**, routed by task id.
- **Session = microVM, 1:1.** A new `runtimeSessionId` boots a fresh Firecracker VM (2 vCPU, 8 GB) from the pristine image; `StopRuntimeSession` terminates and sanitizes it.
- The image is untouched except for one wrapper layer: a standalone Python + a stdlib HTTP server (`exec`, files, boot timestamps). Hidden tests are injected **after** the agent finishes.

Source: `code/demos/demo_cpu_tasks/terminalbench/` deploy; session model per AgentCore docs [D3] (slide 19); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

229 benchmark tasks shrink to 166 oracle-usable ones: 135 train, 31 held out

Table 3 From benchmark to training set

Stage	Tasks	Dropped because
Terminal-Bench tasks	229	—
Deployable (arm64, $\leq$ 2 GB image)	**218**	7 don't build for arm64 · 4 exceed the fixed 2 GB image cap
Oracle-usable	**166**	47 reference solution fails on this substrate · 5 timed out · 0 pass untouched
Train / held-out eval	**135 / 31**	80/20, disjoint by task, stratified by difficulty

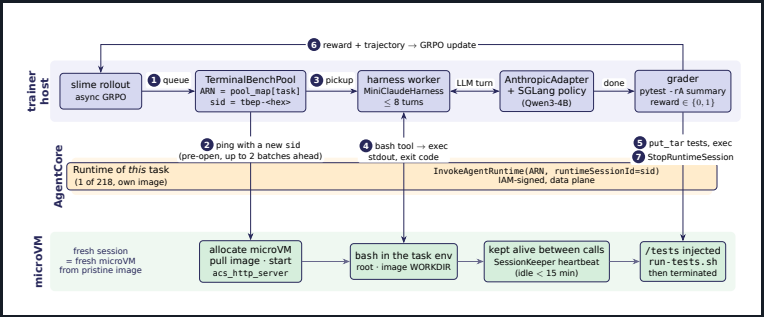
- **Oracle check:** keep a task only if the reference solution passes **and** the untouched environment fails; otherwise its reward is structurally 0 or 1.
- Images: p50 128 MB, p90 492 MB, max 2 GB (48 GB total). The heavy tail (ML models, TeX, toolchains) is what makes boot time vary.
- **Reward = 1 only if every hidden test passes.**

Source: `code/demos/demo_cpu_tasks/terminalbench/` data preparation and oracle check; 2 GB image cap [Q1] (slide 19); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

One training step boots 32 fresh microVMs, one per episode

Figure 2 route → fresh session → agent loop → grade → stop



- There is no “create session” call: the first invoke with a new `runtimeSessionId` **is** the boot.
- One step = RB 4 prompts × NS 8 samples = **32 episodes = 32 fresh microVMs**; 4 harness workers run them.

Source: `code/demos/demo_cpu_tasks/terminalbench/` run config (RB = 4, NS = 8, W = 4); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

On the critical path, fresh sessions would cost ≈ 8 min of serial boot per step

We can't reuse a warm VM. Terminal-Bench tasks change the whole OS (apt, `/etc`, services, `/tmp`). Resetting a workdir would leak state into the next episode and corrupt the reward and the GRPO advantage. **The session is the isolation boundary**, so each episode needs a new one.

Table 4 Cold-start cost

Cost of a fresh session	Typical
Cold boot: allocate VM, pull image, start server	≈ **15 s** for a 447 MB image; grows with image size
Warm hit: AgentCore already had a spare VM up	≈ 0.5 s
Per step, if on the critical path	32 × 15 s ≈ **8 min** of serial boot

- **Bursty:** all 8 GRPO siblings of a task start at once, on the same Runtime.
- **Idle reclaim:** a session silent for ≈ 15 min is reclaimed, and the weight-sync pause can be that long [D3] (slide 19).

Source: `code/demos/demo_cpu_tasks/terminalbench/` session-open timings; idle timeout per [Q1], [D3]; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Only look-ahead pre-open hides the boot, keeps isolation and stays on-policy

- **slime only reveals a few episodes at a time:** at most `CONC` × `engines` = 8 `generate()` calls in flight.
- **It draws step N+1 only after step N finishes**, so every step's first episodes would wait for a boot.
- **slime knows nothing about sandboxes.** Tasks, Runtimes and sessions all live in our `TerminalBenchPool`, inside slime's rollout process.

Table 5 **Options**

Option	Hides boot?	Keeps isolation?	Catch
Reuse one warm session per task + reset	✓	✗	OS state leaks → **rejected**
Standing warm pool per task	✓	✓	218 tasks × NS idle VMs
Open on pickup	✗	✓	Boot on the critical path
Fully-async rollout	✓	✓	Off-policy (stale weights); slime mode change
Look-ahead pre-open (used)	✓	✓	Guesses can be wrong; booted VMs idle

Source: `code/demos/demo_cpu_tasks/terminalbench/` rollout config (CONC, engines); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Four mechanisms hide the boot with no slime change and at most 68 live microVMs

1. **Look-ahead data source.** `tb_lookahead_source.py` (a `--data-source-path` plugin) replays slime's deterministic order (sequential offset, `random.seed(seed + epoch)` shuffle) to list the **next 2 rollouts'** prompts, without moving slime's position. No slime change.
2. **Bounded pre-open window.** The pool keeps a fresh session booted for each of the next **P** = `TB_PREOPEN_BATCHES` × RB × NS = **64** upcoming episodes, pooled per task. A worker claims any ready session for its task; the claim moves the window on by one. Wrong guesses are stopped at the next draw.
3. **Heartbeat.** A `SessionKeeper` pings every open session at half the 15-min idle timeout, so none is reclaimed during training or weight sync.
4. **Retry on a new id.** A failed open retries on a **new** session (3×); an infra failure is logged as `infra_error`, never scored as a wrong answer.

Live microVMs $\leq$ W + P = 4 + 64 = **68**. The account quota is **5000** active sessions and 25 new/s [Q2] (slide 19); throttled opens back off and retry.

Source: `code/demos/demo_cpu_tasks/terminalbench/` (`tb_lookahead_source.py`, `TerminalBenchPool`); quotas [Q2] us-west-2, 2026-09-23; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Across a step boundary no worker waits for a boot; the price is idle VMs

Table 6 **Look-ahead across a step boundary (W = 4, P = 64)**

Moment	What happens	Worker waits for a boot?
Step N−1 running	Its draw already announced steps N and N+1 → 64 sessions booting	—
Step N drawn	Window = steps N, N+1: already booted, nothing new to open	—
Each claim	Next episode of step N+2 starts booting	**No**
Step N training	64 booted VMs idle through training; heartbeat keeps them	—
Step N+1 drawn	All its episodes already booted	**No**

- **Sessions per Runtime are never configured.** A task sampled in a step gets NS = 8 sessions; others get 0.
- **The price is idle time:** each VM is booted about $P \cdot E / W$ seconds before it's needed (E = episode time).

Source: `code/demos/demo_cpu_tasks/terminalbench/` pool design (W = 4, TB_PREOPEN_BATCHES = 2); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

One batch of look-ahead already hides the boot; the run's two cost ≈ 6 min idle per VM

Boot is hidden when $\mathbf{P \cdot E / W \geq B}$ (B = boot time), and a window of at least one batch (P ≥ RB × NS) also covers each step's first episodes. Everything beyond that is paid idle time.

Table 7 **Choosing TB_PREOPEN_BATCHES**

TB_PREOPEN_BATCHES	P	Boot hidden?	Idle per VM ≈ P · E / W
0 — look-ahead off	≈ 4	Mid-step only; each step's first W wait	≈ 20 s
1 — one batch ahead	32	Yes, including step starts	≈ 3 min
2 — used in the smoke run	**64**	Yes, with a full batch of slack	≈ 6 min (measured: 7.3 min p50)

- E = 21 s (measured episode p50), W = 4.
- Tune from the `[demo]` lines: `open_wait_ms` ≫ 0 → raise it; `preopen_idle_ms` ≫ boot time → lower it.

Source: Smoke run 2026-09-23 (E, measured idle), `code/demos/_smoke_logs/lanes_tb_0923`; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

The agent made partial progress (42 %) but solved only 4 of 382 episodes

Table 8 Learning (RB = 4, NS = 8, Qwen3-4B; smoke run, 10 steps + eval, 2026-09-23)

Metric	Value
Train solve rate, steps 1–10	0 · 0 · 0.088 · 0 · 0 · 0 · 0 · 0.026 · 0 · 0
Full solves	**4 / 382** episodes (`acl-permissions-inheritance` ×3, `git-leak-recovery`)
Held-out solve rate (31 tasks × 2)	0.0
Partial progress	**42 %** of episodes pass ≥ 1 hidden test
Agent behaviour	Median 6 turns, 5 bash calls; 45 % hit the 8-turn budget

- **Sparse, not dead.** The tool loop works and the agent makes partial progress, but almost every GRPO group is all-0, so there is little gradient.

Source: Smoke run 2026-09-23, `code/demos/_smoke_logs/lanes_tb_0923`; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

The smoke run hid every boot and lost no episode, but ≈ 95 % of microVM time sat idle

Table 9 **Sandbox and cost (W = 4, TB_PREOPEN_BATCHES = 2)**

Metric	Value
Episodes / infra errors	382, each on its own fresh microVM / **0** lost (job ≈ 70 min)
Boot on the critical path (`open_wait_ms`)	**p50 0 · p95 0**; max 4.9 s (first 4 episodes only)
Session open (`open_ms`)	p50 0.66 s · p95 2.5 s · **100 % warm hits** · 0 retries
Episode time p50 / p90	21 s / 64 s (agent 9 s + grade 10 s)
Pre-opened VM idle p50 / max	**7.3 min** / 16.5 min; heartbeat kept all alive
microVM-seconds per train step	≈ 13 000 (32 × 411 s); **≈ 95 % idle**

- **Every open was a warm hit.** AgentCore already had a spare VM up for each Runtime, so no ≈ 15 s cold boot occurred in this run. The per-image cold-start probe (`probe_coldstart.py`) is still to run.

Source: Smoke run 2026-09-23, `code/demos/_smoke_logs/lanes_tb_0923`; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Pre-open cut the median wait to $\leq$ 5 ms and makespan by 15–20 %; only the first batch waits

Host-only, no GPU or trainer ($\approx$ 7 min, a few cents): the same 24 MockPolicy `sql` episodes in batches of 6, **cold** (open at episode start) vs **preopen** (next batch booted while this one runs, the `TB_PREOPEN_BATCHES` idea).

Table 10 **Pre-open vs cold open (all rewards 1.0)**

Backend	Mode	Wait p50	Episodes that waited	Idle held	Makespan
Own Runtime	cold	774 ms	24 / 24	0 s	31.9 s
Own Runtime	preopen	**5 ms**	8 / 24 (first batch)	75 s	25.5 s (−20 %)
Code Interpreter	cold	1715 ms	24 / 24	0 s	28.7 s
Code Interpreter	preopen	**0 ms**	6 / 24 (first batch)	70 s	24.3 s (−15 %)

- **Only the first batch pays the boot.** The price is “idle held”, the same trade-off `preopen_idle_ms` shows at full scale (slide 14).

Source: Warm-start probe 2026-09-25, `code/demos/demo_cpu_tasks/terminalbench/warmstart/`; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Heartbeat kept 3/3 own-Runtime sessions past the idle timeout; without it, 0/3 survived

A Runtime session idle past `idleRuntimeSessionTimeout` is reclaimed. In training that timeout is the **15-min default** (`WARM_IDLE_TIMEOUT` = 900, pings at 450 s). The probe Runtime `acs_warmstart_cpu` sets it to **60 s** only so the test fits in minutes; sessions then sit idle 150 s.

Table 11 Heartbeat vs idle reclaim

Backend	Heartbeat	Sessions surviving the idle gap	Resume
Own Runtime	on (14 pings)	**3 / 3**	104 ms, files intact
Own Runtime	off	0 / 3	**Silently reclaimed:** same session id, fresh empty sandbox
Code Interpreter	on / off	0 / 3	Ends at its `sessionTimeoutSeconds` regardless of pings

- **Why the keeper is always on here:** a reclaimed session looks healthy but has lost its files. On the Code Interpreter the lever is a generous session timeout, not pings.

Source: Warm-start probe 2026-09-25, `code/demos/demo_cpu_tasks/terminalbench/warmstart/` (Runtime `acs_warmstart_cpu`); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Cold start is solved but over-provisioned; learning signal is now the bottleneck

- **Works end to end:** per-task Runtimes, one fresh microVM per episode, oracle-checked tasks, 0 infra failures.
- **Cold start is solved, but over-provisioned.** Look-ahead removed all boot wait (the warm-start probe confirms: only the first batch waits), but at P = 64 a VM idles ≈ 7 min for a 21 s episode. With warm hits at ≈ 0.7 s, `TB_PREOPEN_BATCHES` **= 1 (or 0)** should give the same wait at a fraction of the cost.
- **Learning signal is the bottleneck:** 4 solves in 382 episodes. Next: easy-first curriculum, more turns, or partial credit from `tests_passed`.

Table 12 **Next steps**

Next	Why
Re-run with `TB_PREOPEN_BATCHES` = 1 and 0	Measure the cost/wait trade-off
`probe_coldstart.py` over all 218 Runtimes	Cold vs warm boot by image size (burst opens)
Curriculum / partial-credit reward	Mixed GRPO groups → real gradient

Source: Smoke run 2026-09-23 and warm-start probe 2026-09-25, `code/demos/demo_cpu_tasks/terminalbench/`; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Appendix

References

AgentCore quotas and docs back every limit used in this deck

Table 13 References

Ref	Source	Supports
[Q1]	AgentCore quotas: docs.aws.amazon.com/bedrock-agentcore/latest/devguide/bedrock-agentcore-limits.html	1000 Runtimes/account · image $\leq$ 2 GB · 15-min request timeout · 15-min idle timeout
[Q2]	`aws service-quotas list-service-quotas --service-code bedrock-agentcore` (us-west-2, 2026-09-23)	Active Session Workloads = 5000 (L-3E5722B2) · new sessions 25/s (L-8EE2AEA2) · image 2048 MB (L-0A9E32B3)
[D3]	docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html	Session = dedicated microVM; 15-min idle, 8 h max lifetime

Source: AWS documentation and service-quotas CLI, us-west-2, 2026-09-23; checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Thank you

Kaige Yang · Senior Applied Scientist, AWS